

WHAT ARE CI/CD PIPELINES?

CI/CD pipelines are the backbone of modern DevOps, designed to automate the process of software delivery. The acronym stands for **Continuous Integration** and **Continuous Deployment** (or **Continuous Delivery**).

The primary goal is to minimize manual intervention, reduce human error, and accelerate the feedback loop between writing code and providing value to users.

1. Continuous Integration (CI)

CI focuses on the early stages of the development cycle. Whenever a developer pushes code to a shared repository (like GitHub or GitLab), the pipeline automatically triggers a series of steps:

- **Build:** The source code is compiled into an executable artifact.
- **Unit Testing:** Small pieces of code are tested in isolation to ensure logic is correct.
- **Static Analysis:** Tools scan the code for security vulnerabilities, style inconsistencies, or potential bugs.

2. Continuous Delivery (CD)

Continuous Delivery ensures that the code is always in a "deployable" state. After the CI stage, the application is automatically deployed to a testing or staging environment.

- **Integration Testing:** Ensures that different modules of the system work together.
- **User Acceptance Testing (UAT):** A final check to ensure the software meets business requirements.
- **Manual Trigger:** In Continuous Delivery, the final push to the live production environment is usually a manual "one-click" decision.

3. Continuous Deployment (CD)

Continuous Deployment is the most advanced stage. It removes the manual trigger from the Delivery phase. Every change that passes all previous stages of the pipeline is automatically released to production without human intervention. This requires high confidence in the automated testing suite.

Core Components of a Pipeline

To function effectively, a CI/CD pipeline relies on several technical components:

- **Source Control:** A central repository (Git) that acts as the single source of truth.

- **Build Tools:** Orchestrators like Jenkins, GitHub Actions, GitLab CI, or Azure Pipelines that execute the steps.
- **Environment Orchestration:** Tools like Docker and Kubernetes that ensure the application runs consistently across different servers.

Key Benefits

Benefit	Description
Speed	Features and bug fixes reach users in minutes or hours rather than weeks.
Reliability	Automated tests catch bugs before they reach production.
Consistency	The deployment process is identical every time, eliminating "it works on my machine" issues.
Visibility	Teams can see exactly where a build failed and why, making it easier to fix.

Limitations and Edge Cases

- **Testing Overhead:** A pipeline is only as good as its tests. Writing and maintaining a robust test suite requires significant engineering effort.
- **Security Risks:** If a pipeline is compromised, an attacker could potentially inject malicious code directly into production.
- **Initial Complexity:** Setting up a fully automated pipeline for legacy monolithic applications can be highly complex and resource-intensive.

How does an application/ a product go live?

The journey of a product from a developer's laptop to a live environment is a multi-stage process known as the **Software Development Life Cycle (SDLC)**. This process is designed to ensure that code is functional, secure, and capable of handling real-world traffic.

1. Development and Local Testing

The process begins with developers writing code. They use **Version Control Systems (VCS)** like Git to manage changes. Before code is even shared, developers run "Local Tests" to ensure their specific feature doesn't break existing functionality.

2. Integration and Build

Once code is pushed to a central repository (e.g., GitHub or GitLab), it enters the **CI/CD Pipeline**.

- **The Build Stage:** The source code is compiled and converted into a deployable format, such as a Docker image or an executable file.
- **Continuous Integration (CI):** Automated scripts run unit tests and "linting" (syntax checking). If any test fails, the build is rejected, preventing "broken" code from moving forward.

3. Staging and Quality Assurance (QA)

Before reaching the general public, the product is deployed to a **Staging Environment**. This is a near-identical clone of the production environment.

- **Integration Testing:** Checking how the new code interacts with databases, APIs, and other services.
- **User Acceptance Testing (UAT):** Stakeholders or QA engineers manually test the product to ensure it meets the original business requirements and "feels" right for the user.

4. Infrastructure Provisioning

Modern products aren't just code; they require "houses" to live in. Using **Infrastructure as Code (IaC)** tools like Terraform or CloudFormation, teams automatically set up the necessary cloud resources (servers, databases, load balancers, and security groups) in AWS, Azure, or Google Cloud.

5. The Release (Deployment Strategies)

Going "live" is rarely as simple as flipping a single switch. Engineering teams use specific strategies to minimize risk:

- **Blue-Green Deployment:** Two identical environments exist. "Green" is live; "Blue" is the new version. Once Blue is verified, traffic is instantly routed from Green to Blue.
- **Canary Release:** The new version is rolled out to a tiny percentage of users (e.g., 5%) first. If no errors are detected, it is gradually rolled out to everyone else.
- **Rolling Deployment:** Instances are updated one by one, ensuring there is always a version of the app running to prevent downtime.

6. Monitoring and Maintenance

Once the product is live, the process doesn't end. Teams use **Observability tools** (like Prometheus or Datadog) to monitor:

- **Performance:** Is the app slow?
- **Errors:** Are users seeing 404 or 500 error pages?
- **Traffic:** Is the server handling the load, or does it need to "auto-scale"?

Comparison of Deployment Environments

Environment	Purpose	Access
Development	Writing and testing raw code	Developers only
QA/Staging	Final testing and bug hunting	QA Team / Stakeholders
Production	Actual usage	The Public / End Users

What are different environments while working on a project?

In professional software development, environments are isolated "tiers" or workspaces where code lives at different stages of its maturity. Using multiple environments prevents unfinished features or bugs from reaching the end user.

The standard progression follows a path of increasing stability and restricted access.

1. Local Environment

This is the developer's individual computer. It is the only place where it is "safe" for the code to be broken.

- **Purpose:** Active coding, experimenting, and initial debugging.
- **Tools:** IDEs (VS Code, PyCharm), local databases (Docker containers), and compilers.
- **Data:** Usually "mock" data or a small subset of anonymized records.

2. Development (Dev) Environment

The first shared environment where the team's code is integrated.

- **Purpose:** To see how different features developed by different people work together.
- **Stability:** Low. Since code is pushed here frequently, features may be partially broken.
- **Focus:** Integration testing and basic functionality checks.

3. Testing / QA (Quality Assurance)

A stabilized environment dedicated to finding bugs before the software moves toward the user.

- **Purpose:** Rigorous testing by QA engineers.
- **Tests performed:** Regression testing (making sure new changes didn't break old features), security scanning, and performance testing.
- **Data:** Often uses a "cleaned" copy of production data to simulate real-world scenarios.

4. Staging (Pre-Production)

Staging is a **mirrored copy** of the live Production environment. It is the final "dress rehearsal."

- **Purpose:** To perform **User Acceptance Testing (UAT)** and verify that the deployment process itself works.
- **Configuration:** The hardware, database versions, and network settings should be identical to the live site.
- **Rule:** If a bug is found here, the release is usually halted until it is fixed.

5. Production (Prod)

This is the "Live" environment that your customers actually use.

- **Purpose:** To serve the end users.
- **Access:** Extremely restricted. Developers usually cannot change code here directly; changes must come through a verified CI/CD pipeline.
- **Monitoring:** High-level alerts are set up here to notify engineers immediately if the system goes down or slows down.

Summary of Environment Hierarchy

Environment	Who uses it?	Criticality	Data Type
Local	Individual Developer	Low	Dummy/Mock
Dev	Engineering Team	Medium	Synthetic
QA	Testers / QA Engineers	High	Anonymized/Sample
Staging	Stakeholders / Clients	Critical	Mirror of Prod
Production	End Users	Absolute	Real User Data

Special Case Environments

- **Sandbox:** Often provided to external third-party developers (like a payment gateway) to test their integrations without affecting any real accounts.
 - **DR (Disaster Recovery):** A standby environment in a different geographic region used only if the primary Production environment suffers a massive failure.
-

Notes:

Continuous integration:

– We check in the code – Build should happen – Artifact is created – package

Continuous deployment:

– Package of CI – deployed in the azure VM or server or web app

Azure DevOps Main Services

Service	Purpose
Azure Boards	Project management and tracking
Azure Repos	Source code management using Git
Azure Pipelines	CI/CD automation
Azure Test Plans	Manual and automated testing
Azure Artifacts	Package management

1. Azure Boards

Used for Agile project management.

Features

- Work item tracking
- Sprint planning
- Backlogs
- Kanban boards
- Task assignment
- Bug tracking

Hierarchy Structure

Epic -> Feature -> User Story -> Task

Example

Epic: Customer Service

Feature: Returns and Refunds

User Story: User can return products

Task: Create return API

2. Azure Repos

Azure Repos provides Git repositories for source code management.

Features

- Git repositories
- Branching
- Pull requests
- Code reviews
- Version control
- Collaboration

Common Git Flow

Main Branch -> Feature Branch -> Bugfix Branch -> Release Branch

3. Azure Pipelines

Azure Pipelines automates:

- Build process
- Testing
- Deployment

Supports:

- Java
 - Python
 - .NET
 - Node.js
 - Docker
 - Kubernetes
-

CI/CD in Azure Pipelines

Continuous Integration (CI)

The developer checks in code to the repository.

Pipeline automatically:

1. Builds application
2. Runs tests
3. Creates artifact/package

Flow

Code Commit -> Build -> Test -> Artifact Created

Continuous Deployment (CD)

After CI succeeds:

1. Artifact/package is deployed
2. Application moves to target environment

Flow

Artifact -> Deploy -> Azure VM / Web App / Server

Development Environment (Dev Environment)

Used by developers for coding and initial testing.

Activities

- Develop features
- Run local tests
- Fix development issues

QA Environment

Used by QA/Test Engineers.

Activities

- Application testing
- File bugs/issues
- Developers fix bugs
- Re-testing happens

Azure Pipeline Types

Pipeline Type	Purpose
Build Pipeline	Compiles and tests code
Release Pipeline	Deploys application
YAML Pipeline	Pipeline defined as code

Azure Artifacts

Used to store:

- Packages
- Dependencies
- Libraries

Supports:

- Maven
- npm

- NuGet
 - Python packages
-

Advantages of Azure DevOps

- Faster software delivery
 - Automation reduces manual work
 - Better collaboration
 - Easy bug tracking
 - Continuous testing
 - Scalable cloud integration
 - Supports Agile and Scrum
-

Basic CI/CD Workflow

1. Developer Writes Code
2. Push to Azure Repo
3. CI Pipeline Triggered
4. Build + Test
5. Artifact Created
6. CD Pipeline Triggered
7. Deploy to Azure VM/Web App
8. QA Testing
9. Production Release